

GitHub Copilot CLI: Instalar, entender y auditar a tu agente

Entorno de ejecución y preparación del repositorio (PowerShell 7 y Git)

¿Qué es el concepto?

Un agente de desarrollo con IA no solo responde mis preguntas, sino que ejecuta comandos, inspecciona mis repositorios y edita código de forma autónoma en un ciclo iterativo. Para trabajar de manera segura en mi equipo con Windows 11, comprobé que necesito tener un entorno moderno como PowerShell 7 (configurado con la política RemoteSigned), ya que la CLI se ejecuta mediante scripts y el agente delega sus acciones directamente en esta shell. Finalmente, siempre debo trabajar en una copia aislada del repositorio; de esta forma evito comprometer mi código base, filtrar secretos (como mis archivos .env, llaves .pem).

El proceso que realicé:

Configuré la terminal predeterminada en PowerShell 7 dentro de Windows Terminal, verificando la versión con `$PSVersionTable.PSVersion.Major`. Para aislar mi espacio de trabajo sin tocar AcademiaExtras, ejecuté `git archive` para extraer únicamente los archivos limpios y versionados de `taskflow-api` hacia un archivo comprimido temporal en `$HOME`. Descomprimí el proyecto en mi nueva carpeta `taskflow-copilot-lizinzunza934`, creé un archivo `.gitignore` estricto y verifiqué con `git ls-files` y expresiones regulares que ningún archivo sensible estuviera en staging antes del commit inicial. Posteriormente, creé el repositorio público y vacío en GitHub, enlacé el origen remoto y subí la rama `main`. Por último, configuré Apache Maven 3.9.9 en mis variables de entorno nativas de Windows para validar la suite de pruebas unitarias con `mvn -q clean test`, confirmando que el proyecto partiera de 67 tests en verde y 0 fallos.

```
Windows PowerShell
PS C:\Users\User\taskflow-copilot-lizinzunza934> git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
PS C:\Users\User\taskflow-copilot-lizinzunza934> git log --oneline -n 3
d77cb3c (HEAD -> main, origin/main) Día 1: ARQUITECTURA.md verificado y evidencia
9dd19a Instrucciones de Copilot del curso
768630d TaskFlow API, punto de partida
PS C:\Users\User\taskflow-copilot-lizinzunza934> mvn -q test
10:14:17.323 [main] INFO org.springframework.test.context.support.AnnotationConfigContextLoaderUtils -- Could not detect default configuration class
es for test class [com.taskflow.integration.AuthControllerTest]: AuthControllerTest does not declare any static, non-private, non-final, nested clas
ses annotated with @Configuration.
10:14:17.900 [main] INFO org.springframework.boot.test.context.SpringBootTestContextBootstrapper -- Found @SpringBootConfiguration com.taskflow.Task
flowApiApplication for test class com.taskflow.integration.AuthControllerTest
2026-09-15T10:14:19.586-06:00 INFO 4256 --- [taskflow-api] [ main] c.t.integration.AuthControllerTest : Starting AuthControllerTest
using Java 21.0.12 with PID 4256 (started by User in C:\Users\User\taskflow-copilot-lizinzunza934)
2026-09-15T10:14:19.589-06:00 INFO 4256 --- [taskflow-api] [ main] c.t.integration.AuthControllerTest : The following 1 profile is
active: "test"
2026-09-15T10:14:22.234-06:00 INFO 4256 --- [taskflow-api] [ main] .s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data J
PA repositories in DEFAULT mode.
2026-09-15T10:14:22.391-06:00 INFO 4256 --- [taskflow-api] [ main] .s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data reposi
tories
Hibernate: insert into projects (created_at,description,name,owner_id,id) values (?, ?, ?, ?, default)
Hibernate: select pl_0.id,pl_0.created_at,pl_0.description,pl_0.name,pl_0.owner_id from projects pl_0 where pl_0.id=?
Hibernate: insert into tasks (assignee_id,description,due_date,priority,project_id,status,title,id) values (?, ?, ?, ?, ?, ?, default)
Hibernate: select t1_0.id,t1_0.assignee_id,t1_0.description,t1_0.due_date,t1_0.priority,t1_0.project_id,t1_0.status,t1_0.title from tasks t1_0 where
t1_0.id=?
Hibernate: select t1_0.id,t1_0.assignee_id,t1_0.description,t1_0.due_date,t1_0.priority,t1_0.project_id,t1_0.status,t1_0.title from tasks t1_0 where
t1_0.id=?
PS C:\Users\User\taskflow-copilot-lizinzunza934> $r = Select-String -Path target\surefire-reports\*.txt -Pattern 'Tests run: (\d+), Failures: (\d+),
Errors: (\d+)'
PS C:\Users\User\taskflow-copilot-lizinzunza934> "tests: $($r | ForEach-Object { [int]$_Matches[0].Groups[1].Value } | Measure-Object -Sum).Sum" ·
fallos: 0"
tests: 67 · fallos: 0
PS C:\Users\User\taskflow-copilot-lizinzunza934> |
```

Ilustración 1 - Configuración de Git, push inicial al repositorio personal y validación de los 67 tests en verde.

Comentarios personales:

Me di cuenta de lo importante de usar git archive en lugar de solo copiar las carpetas. Al revisar el directorio previo de AcademiaExtras, tenía carpetas locales como .idea/ o target/ que habrían ensuciado el historial o subido datos que no necesito a GitHub. Aprender a contar los tests desde PowerShell en lugar de pedirle al modelo que lo haga me enseñó a no darle a la IA tareas que se resuelven en segundos con un comando, ahorramos tokens que siempre es mejor.

Modelo, control de costos y auditoría de alucinaciones

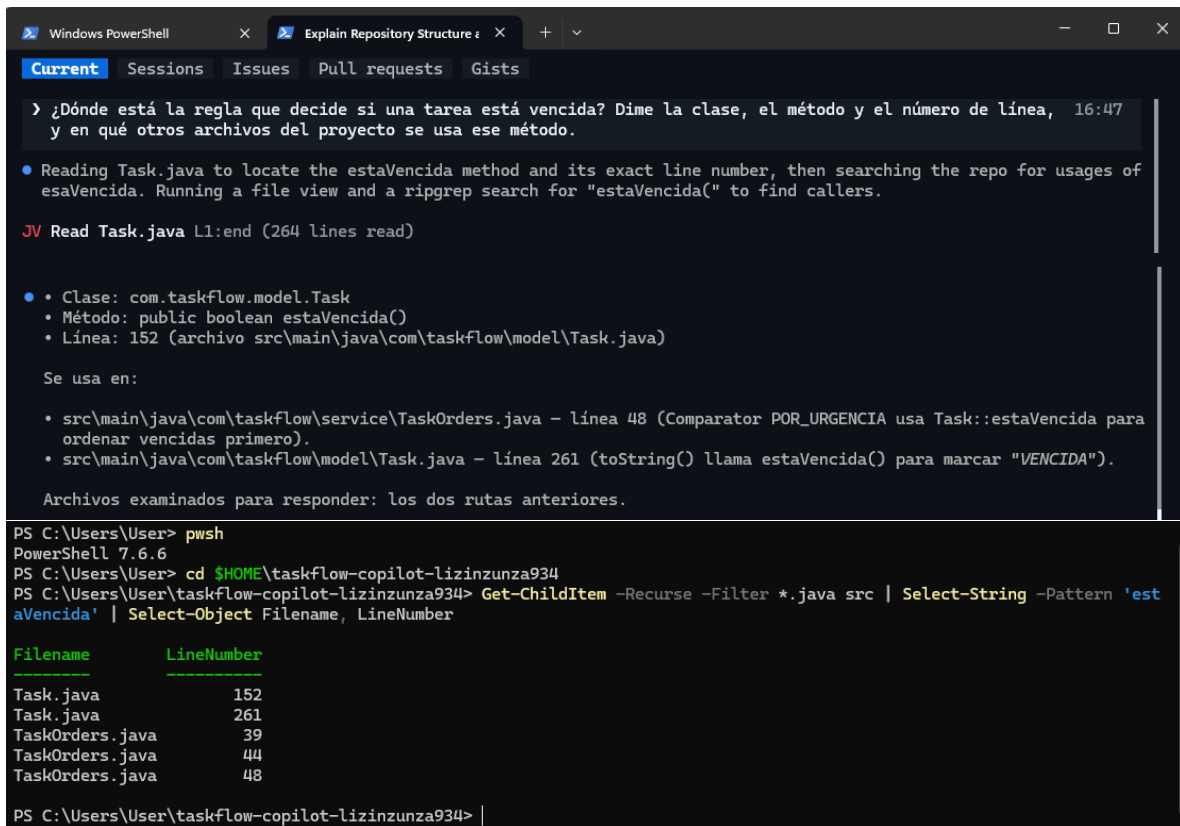
¿Qué es el concepto?

Los modelos de lenguaje no leen mis archivos gratis, procesan tokens dentro de una ventana de contexto y cada consulta que hago consume créditos (AI Credits). Si el modelo no tiene el archivo exacto en su contexto, me genera texto que suena convincente pero puede ser falso. Por ello, la regla fundamental al trabajar con un agente es comprobar lo que me dice con comandos independientes del sistema (Test-Path, Select-String, mvn test), y no confiar en sus afirmaciones sin comprobar.

El proceso que realicé:

Fijé globalmente el modelo gpt-5-mini estableciendo la variable de entorno COPILOT_MODEL a nivel de usuario para optimizar los 1,500 créditos mensuales de mi suscripción Copilot Pro. Verifiqué dentro de la CLI con los comandos /model y /usage que el modelo estuviera activo con 0 créditos consumidos al iniciar. Luego, trabajé con dos pestañas simultáneas: la pestaña 1 con el chat de Copilot y la pestaña 2 con PowerShell 7. Realicé tres preguntas de prueba al agente y verifiqué sus respuestas:

- Comprobé la estructura de paquetes citada por el agente contrastándola contra la lista real de 10 carpetas obtenida con `Get-ChildItem src\main\java\com\taskflow -Directory`.
- Verifiqué la ubicación de la regla estaVencida corriendo `Select-String` en todos los .java, confirmando que vive en Task.java y TaskOrders.java.
- Le pregunté por un endpoint inexistente de tareas vencidas; al inspeccionar los 16 endpoints reales de los controladores con expresiones regulares, confirmé que la API no cuenta con esa ruta.



```
Windows PowerShell
X Explain Repository Structure
+ - v

Current Sessions Issues Pull requests Gists

> ¿Dónde está la regla que decide si una tarea está vencida? Dime la clase, el método y el número de línea, 16:47
y en qué otros archivos del proyecto se usa ese método.

• Reading Task.java to locate the estaVencida method and its exact line number, then searching the repo for usages of
estaVencida. Running a file view and a ripgrep search for "estaVencida(" to find callers.

JV Read Task.java L1:end (264 lines read)

• Clase: com.taskflow.model.Task
• Método: public boolean estaVencida()
• Línea: 152 (archivo src/main/java/com/taskflow/model/Task.java)

Se usa en:

• src/main/java/com/taskflow/service/TaskOrders.java - línea 48 (Comparator POR_URGENCIA usa Task::estaVencida para
ordenar vencidas primero).
• src/main/java/com/taskflow/model/Task.java - línea 261 (toString() llama estaVencida() para marcar "VENCIDA").

Archivos examinados para responder: los dos rutas anteriores.

PS C:\Users\User> pwsh
PowerShell 7.6.6
PS C:\Users\User> cd $HOME\taskflow-copilot-lizinzunza934
PS C:\Users\User\taskflow-copilot-lizinzunza934> Get-ChildItem -Recurse -Filter *.java src | Select-String -Pattern 'est
aVencida' | Select-Object Filename, LineNumber

Filename      LineNumber
-----
Task.java      152
Task.java      261
TaskOrders.java 39
TaskOrders.java 44
TaskOrders.java 48

PS C:\Users\User\taskflow-copilot-lizinzunza934> |
```

Ilustración 2 - Consulta interactiva en Copilot CLI y contrastación de alucinaciones con comandos en PowerShell.

Comentarios personales:

Comprobar que la IA puede hablar con seguridad de métodos o rutas que no existen me hace darme cuenta de lo importante que es comprbar todo. El ejercicio de contrastar con Test-Path y Select-String me deja claro que el agente es un asistente, pero el criterio técnico y la validación final siempre dependen del desarrollador.

Permisos del agente e instrucciones del proyecto (copilot-instructions.md)

¿Qué es el concepto?

El agente cuenta con herramientas de sistema que pueden modificar el disco o ejecutar procesos. Para prevenir que haga acciones destructivas o accidentales, la CLI me pide confirmación mediante barreras de aprobación que escojo (Yes, Yes and don't ask again, No). El archivo .github/copilot-instructions.md funciona como un prompt de sistema que el agente lee obligatoriamente cuando inicio cualquier sesión, estableciendo mis reglas de codificación, perfiles de Maven (H2), el idioma en el que responde y sus límites operativos.

El proceso que realicé:

Evalué las tres políticas de permisos dentro de Copilot CLI:

- Aprobé una ejecución de lectura de mvn -q test.

- Rechacé una orden destructiva cuando le pedí borrar la carpeta target, seleccionando la opción negativa para indicarle que no ejecutara comandos administrativos.
- Evalué la modificación y reversión de archivos: le pedí agregar una línea a README.md, revisé la diferencia exacta con /diff, y posteriormente utilicé /rewind seleccionando Conversation + files para restaurar el archivo en disco, validando con git status --short que el repositorio quedara limpio.

Posteriormente, copié el archivo oficial de instrucciones del curso desde \$HOME\AcademiaExtras\copilot\dia-1\copilot-instructions.md hacia .github\copilot-instructions.md, reinicié la sesión de Copilot y verifiqué su carga automática mediante el comando /instructions, subiendo el cambio a GitHub con su respectivo commit.

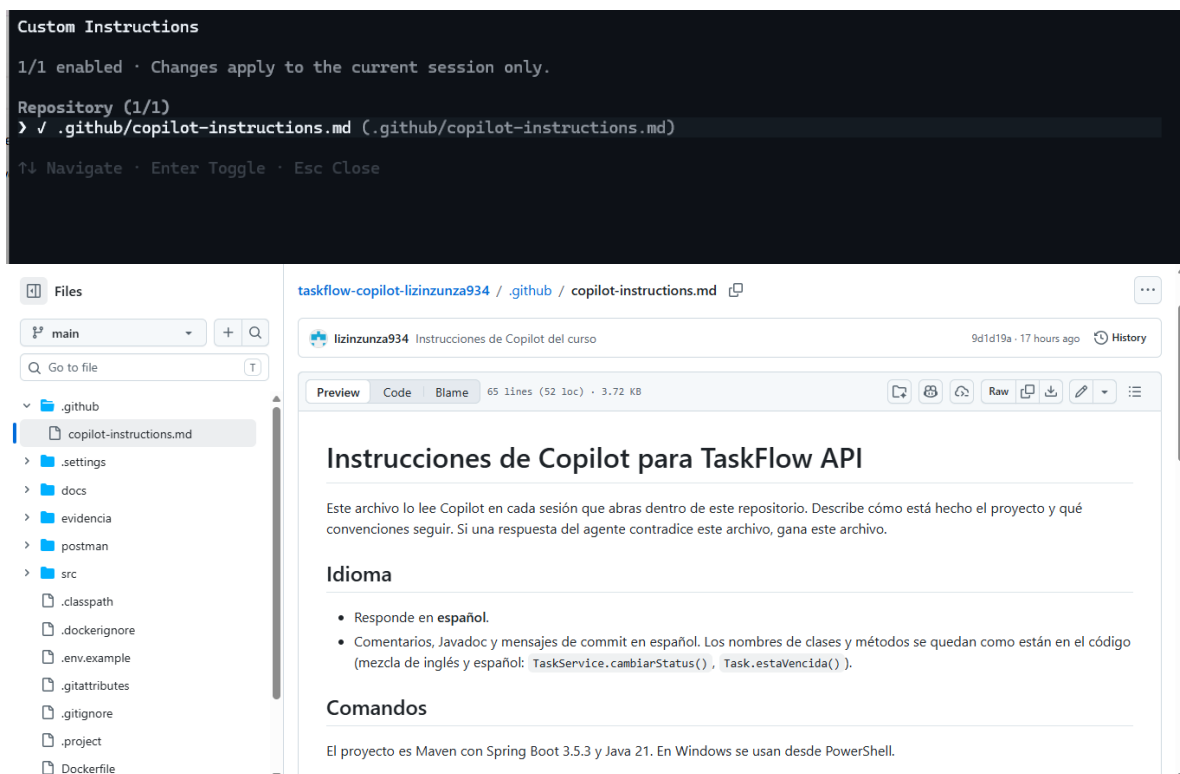


Ilustración 3 - Gestión de permisos interactivos y carga del archivo de directrices del proyecto.

Comentarios personales:

Probar el comando de borrado me enseñó a leer detenidamente antes de dar un Enter. El agente puede sugerir comandos invasivos de sistema para saltarse restricciones; negárselo y saber cómo usar /rewind para regresar el disco al estado original da mucha confianza para experimentar sin miedo a romper el entorno.

Integrador: Generación y auditoría de docs/ARQUITECTURA.md

¿Qué es el concepto?

En proyectos reales, la documentación técnica generada por IA debo auditarla antes de integrarla. La técnica de auditoría estática que utilizo consiste en extraer mediante scripts deterministas todas las entidades citadas en el documento (clases, rutas de archivos, métodos, endpoints) y validar si existen físicamente en mi código. Si encuentro discrepancias, retroalimentación al agente pasándole la evidencia exacta para que corrija sus propios errores, sin desperdiciar mis tokens relejendo todo el código.

El proceso que realicé:

Inicié una sesión de Copilot rastreando el consumo de tokens con el parámetro `--usage-output-file evidencia\dial\uso-integrador.json` y le ordené redactar docs/ARQUITECTURA.md bajo lineamientos estrictos de sintaxis. Al correr por primera vez el script de auditoría verificar-arquitectura.ps1, el reporte detectó 7 elementos con la etiqueta [NO EXISTE] (rutas en carpetas incorrectas como GlobalExceptionHandler.java dentro de exception/ en vez de advice/, métodos inventados como TaskService.create y Task.asignarResponsable, y clases inexistentes).

Exporté la salida del verificador a evidencia\dial\verificador.txt mediante Tee-Object y le pedí a Copilot que leyera únicamente ese archivo para corregir las líneas señaladas. En la primera iteración corrigió 6 de los 7 errores; tras una segunda pasada puntual sobre la línea restante, el script verificador confirmó 0 NO EXISTE sobre un total de más de 50 entidades válidas del proyecto. Finalmente, cerré la sesión con `/exit`, transformé el JSON de consumo en un reporte legible (usage.txt), generé el registro de la versión instalada y realicé el push final a la rama principal de mi repositorio.

```
Resumen: 53 OK · 3 REVISAR · 3 EXTERNA · 0 NO EXISTE · 19 sin verificar
PS C:\Users\User\taskflow-copilot-lizinzunza934> Get-ChildItem evidencia\dial

Directorio: C:\Users\User\taskflow-copilot-lizinzunza934\evidencia\dial

Mode                LastWriteTime         Length Name
----                -
-a---            14/09/2026 05:43 p. m.             72 copilot-version.txt
-a---            14/09/2026 05:43 p. m.             55 usage.txt
-a---            14/09/2026 05:43 p. m.            2226 uso-integrador.json
-a---            14/09/2026 05:43 p. m.            8490 verificador.txt

PS C:\Users\User\taskflow-copilot-lizinzunza934> Get-Content evidencia\dial\usage.txt
AI Credits del integrador: 7.46 · modelo: gpt-5-mini
PS C:\Users\User\taskflow-copilot-lizinzunza934> |
```

Ilustración 4 - Auditoría del documento de arquitectura y generación de evidencias de consumo.

Comentarios personales:

El contraste entre la primera y la última corrida del verificador demostró lo fácil que es pasar por alto errores sutiles en un documento generado por IA si solo se lee por encima. Pasar el archivo de reporte (verificador.txt) directo al agente en lugar de pedirle "revisa si te equivocaste" ahorró una enorme cantidad de créditos y resolvió las fallas.